Sesión 12
SISTEMAS DE RECOMENDACIÓN

Filtrado colaborativo, factorización matricial y métricas de evaluación

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **2**

1. Sistemas de Recomendación **3**
1.1. Logro de la sesión . . . 3
1.2. Introducción: la revolución de la personalización . . . 3
1.3. Tipos de sistemas de recomendación . . . 4
1.3.1. Sistemas basados en popularidad . . . 4
1.3.2. Sistemas basados en contenido (Content-based) . . . 4
1.3.3. Sistemas colaborativos (Collaborative Filtering) . . . 4
1.3.4. Sistemas híbridos . . . 5
1.4. Factorización matricial: SVD y Funk SVD . . . 5
1.4.1. Formulación del problema . . . 5
1.4.2. SVD tradicional . . . 5
1.4.3. Funk SVD (Simon Funk) . . . 6
1.4.4. Extensiones: sesgos y factores temporales . . . 6
1.5. Métricas de evaluación en sistemas de recomendación . . . 6
1.5.1. Métricas para predicción de rating . . . 6
1.5.2. Métricas para evaluación de rankings . . . 7
1.6. Problemas y desafíos en sistemas de recomendación . . . 8
1.6.1. Cold start . . . 8
1.6.2. Escalabilidad . . . 8
1.6.3. Matriz dispersa (sparse data) . . . 9
1.6.4. Diversidad y serendipia . . . 9
1.7. Caso integrado: recomendación de películas con MovieLens . . . 9
1.7.1. Datos . . . 9
1.7.2. Preprocesamiento . . . 9
1.7.3. Modelos implementados . . . 9
1.7.4. Métricas de evaluación . . . 9
1.7.5. Resultados típicos . . . 9
1.8. Implementación en Python . . . 10
1.8.1. Filtrado colaborativo con Surprise . . . 10
1.8.2. Funk SVD desde cero con NumPy . . . 10
1.8.3. Cálculo de Precision@k . . . 11
1.9. Aplicaciones en la industria . . . 11
1.9.1. Netflix . . . 11
1.9.2. Amazon . . . 11
1.9.3. Spotify . . . 11
1.9.4. TikTok e Instagram . . . 11

1.10. Conexión con el resto del curso 11
1.11. Resumen 12

Capítulo 1

Sistemas de Recomendación

Los sistemas de recomendación se han convertido en el núcleo de las plataformas digitales modernas. Desde Netflix sugiriendo películas, Spotify creando listas personalizadas, Amazon recomendando productos, hasta TikTok decidiendo qué vídeo mostrar a continuación, todos estos sistemas comparten un objetivo común: predecir qué ítems serán más relevantes para un usuario en particular [citation:4]. Esta sesión aborda los fundamentos matemáticos, los algoritmos más utilizados, las métricas de evaluación y los desafíos prácticos de estos sistemas.

1.1. Logro de la sesión

Diseñar e implementar sistemas de recomendación basados en filtrado colaborativo y factorización matricial, comprendiendo las métricas de evaluación y los desafíos como el cold start y la escalabilidad.

1.2. Introducción: la revolución de la personalización

Antes del auge del machine learning, las recomendaciones se basaban principalmente en rankings de popularidad: los "más vendidos" o "más vistos" se mostraban a todos los usuarios por igual [citation:6]. Aunque esta estrategia aún tiene cabida (por ejemplo, en librerías con su sección de bestsellers), carece de personalización. Un usuario que ha visto muchas películas de terror no encontrará relevante una lista de comedias románticas populares.

Los sistemas de recomendación modernos resuelven este problema utilizando dos tipos de datos fundamentales [citation:4]:

- **Datos explícitos**: información que el usuario proporciona directamente, como valoraciones (1-5 estrellas), likes o comentarios.
- **Datos implícitos**: derivados del comportamiento del usuario, como historial de clics, tiempo de visualización, compras realizadas o canciones escuchadas.

El impacto de estos sistemas es enorme: más del 35 % de las ventas de Amazon provienen de su motor de recomendaciones, y Netflix ha logrado reducir su tasa de cancelación en más de un 20 % gracias a la personalización [citation:9].

1.3. Tipos de sistemas de recomendación

1.3.1. Sistemas basados en popularidad

Son los más simples: recomiendan los mismos ítems a todos los usuarios basándose en métricas globales como número de ventas, visualizaciones o valoraciones promedio [citation:6]. No requieren personalización y son útiles como punto de partida cuando no hay datos históricos (cold start del sistema).

1.3.2. Sistemas basados en contenido (Content-based)

Analizan las características de los ítems que un usuario ha valorado positivamente y recomiendan ítems similares en contenido [citation:4][citation:6]. Por ejemplo, si un usuario ha visto muchas películas de ciencia ficción, el sistema recomendará otras películas del mismo género.

Fundamento matemático: Cada ítem i se representa mediante un vector de características $\mathbf{x}_i \in \mathbb{R}^d$ (por ejemplo, género, director, actores). El perfil del usuario u se construye como el promedio (o suma ponderada) de los vectores de los ítems que ha valorado positivamente:

$$\mathbf{p}_u = \frac{1}{|I_u^+|} \sum_{i \in I_u^+} \mathbf{x}_i$$

La relevancia de un nuevo ítem j para el usuario u se calcula mediante similitud de coseno:

$$\text{relevancia}(u, j) = \frac{\mathbf{p}_u \cdot \mathbf{x}_j}{\|\mathbf{p}_u\| \|\mathbf{x}_j\|}$$

Ventajas: no sufre de cold start para nuevos ítems (si se conocen sus características), es interpretable. **Desventajas:** requiere características bien definidas, tiende a recomendar ítems muy similares (falta de diversidad).

1.3.3. Sistemas colaborativos (Collaborative Filtering)

Se basan en el principio de que usuarios con gustos similares en el pasado tendrán gustos similares en el futuro [citation:6][citation:7]. Existen dos variantes principales.

Filtrado colaborativo basado en usuarios (User-based)

Para un usuario activo u, se encuentran los k usuarios más similares (vecinos) según sus valoraciones. La predicción para un ítem i no valorado por u se calcula como el promedio ponderado de las valoraciones de los vecinos.

Medida de similitud: correlación de Pearson entre vectores de valoraciones de dos usuarios a y b:

$$\text{sim}(a, b) = \frac{\sum_{i \in I_{ab}} (r_{a,i} - \bar{r}_a)(r_{b,i} - \bar{r}_b)}{\sqrt{\sum_{i \in I_{ab}} (r_{a,i} - \bar{r}_a)^2} \sqrt{\sum_{i \in I_{ab}} (r_{b,i} - \bar{r}_b)^2}}$$

donde I_{ab} son los ítems valorados por ambos usuarios, y $\bar{r}_a$ es la valoración media del usuario a.

Predicción:

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|}$$

Filtrado colaborativo basado en ítems (Item-based)

Análogo al anterior, pero se calcula la similitud entre ítems. Para predecir la valoración de un usuario u sobre un ítem i, se buscan ítems similares a i que u haya valorado [citation:1].

Ventaja: la similitud entre ítems es más estable que entre usuarios, por lo que puede precomputarse.

1.3.4. Sistemas híbridos

Combinan enfoques basados en contenido y colaborativos para mitigar las limitaciones de cada uno [citation:4]. Por ejemplo, Netflix utiliza un sistema híbrido que analiza tanto las valoraciones de los usuarios como las características de las películas.

1.4. Factorización matricial: SVD y Funk SVD

La factorización matricial se popularizó gracias al Premio Netflix de 1 millón de dólares en 2006. El objetivo era superar en un 10 % el algoritmo de recomendación de Netflix (que tenía un RMSE de 0.9514) [citation:1].

1.4.1. Formulación del problema

Se tiene una matriz de valoraciones $\mathbf{R}$ de tamaño $m \times n$ (usuarios $\times$ ítems), con muchas celdas vacías (matriz dispersa). El objetivo es encontrar dos matrices $\mathbf{P}$ (usuarios $\times$ factores) y $\mathbf{Q}$ (ítems $\times$ factores) tales que:

$$\mathbf{R} \approx \mathbf{P}\mathbf{Q}^T$$

donde cada fila $\mathbf{p}_u$ representa los factores latentes del usuario u (sus gustos en dimensiones no observables como "preferencia por películas con acción y drama"), y cada fila $\mathbf{q}_i$ representa los factores latentes del ítem i. La valoración predicha es el producto punto:

$$\hat{r}_{u,i} = \mathbf{p}_u \cdot \mathbf{q}_i = \sum_{f=1}^{k} p_{u,f}\, q_{i,f}$$

1.4.2. SVD tradicional

La descomposición en valores singulares (SVD) clásica factoriza la matriz completa, pero tiene limitaciones [citation:1]:

- Requiere pre-llenar los valores faltantes (lo que introduce sesgo).
- Es computacionalmente costosa ($O(m^2 n)$).
- No escala bien a matrices dispersas.

1.4.3. Funk SVD (Simon Funk)

Simon Funk propuso una aproximación mediante optimización que revolucionó el campo [citation:1]. En lugar de descomponer la matriz completa, se optimiza solo sobre las valoraciones conocidas mediante gradiente descendente estocástico (SGD).

Función de pérdida con regularización:

$$\mathcal{L} = \sum_{(u,i)\in\mathcal{K}} (r_{u,i} - \mathbf{p}_u \cdot \mathbf{q}_i)^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

donde $\mathcal{K}$ es el conjunto de pares (usuario, ítem) con valoración conocida, y λ controla la regularización para evitar overfitting.

Actualización SGD: Para cada valoración conocida $r_{u,i}$:

$$\begin{aligned} e_{u,i} &= r_{u,i} - \mathbf{p}_u \cdot \mathbf{q}_i \\ \mathbf{p}_u &\leftarrow \mathbf{p}_u + \eta(e_{u,i}\mathbf{q}_i - \lambda\mathbf{p}_u) \\ \mathbf{q}_i &\leftarrow \mathbf{q}_i + \eta(e_{u,i}\mathbf{p}_u - \lambda\mathbf{q}_i) \end{aligned}$$

donde η es la tasa de aprendizaje.

Ventajas de Funk SVD [citation:1]:

- Funciona excelentemente con matrices dispersas.
- La regularización controla el overfitting.
- Es computacionalmente eficiente y escalable.

1.4.4. Extensiones: sesgos y factores temporales

Los modelos modernos añaden términos de sesgo (bias) para capturar efectos globales:

$$\hat{r}_{u,i} = \mu + b_u + b_i + \mathbf{p}_u \cdot \mathbf{q}_i$$

donde μ es la media global, b_u el sesgo del usuario (usuarios que tienden a valorar más alto o más bajo), y b_i el sesgo del ítem (ítems generalmente mejor valorados). También se incorporan factores temporales para capturar cambios en los gustos a lo largo del tiempo.

1.5. Métricas de evaluación en sistemas de recomendación

La evaluación depende del objetivo: predecir valoraciones numéricas o generar listas de recomendaciones ordenadas.

1.5.1. Métricas para predicción de rating

- **RMSE (Root Mean Square Error)**:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{K}|} \sum_{(u,i)\in\mathcal{K}} (r_{u,i} - \hat{r}_{u,i})^2}$$

  Penaliza más los errores grandes. Fue la métrica oficial del Premio Netflix [citation:1].

- **MAE (Mean Absolute Error)**:

$$\text{MAE} = \frac{1}{|\mathcal{K}|} \sum_{(u,i)\in\mathcal{K}} |r_{u,i} - \hat{r}_{u,i}|$$

  Menos sensible a outliers que RMSE.

1.5.2. Métricas para evaluación de rankings

Precision@k y Recall@k

Miden la calidad de una lista ordenada de k recomendaciones [citation:2][citation:5][citation:8].

- **Precision@k**: proporción de ítems relevantes dentro de las top-k recomendaciones:

$$\text{Precision@k} = \frac{\text{número de ítems relevantes en top-}k}{k}$$

  Responde: "de lo que recomendé, ¿cuánto era relevante?"

- **Recall@k**: proporción de ítems relevantes totales que fueron capturados en las top-k recomendaciones:

$$\text{Recall@k} = \frac{\text{número de ítems relevantes en top-}k}{\text{número total de ítems relevantes para el usuario}}$$

  Responde: "de lo relevante, ¿cuánto logré recomendar?"

Ejemplo numérico: [citation:5][citation:8] Un usuario tiene 6 películas relevantes en su historial. Dos algoritmos recomiendan 3 películas cada uno:

- Algoritmo A: [Terminator, James Bond, Love Actually] $\rightarrow$ 2 relevantes.
- Algoritmo B: [Cars, Toy Story, Iron Man] $\rightarrow$ 1 relevante.

Precision@3: A = 2/3 = 0.667, B = 1/3 = 0.333. Recall@3: A = 2/6 = 0.333, B = 1/6 = 0.167.

Compromiso Precision-Recall: existe una relación inversa. Si se recomiendan muchos ítems, el recall aumenta pero la precisión disminuye [citation:2].

F-score@k

Media armónica que combina precisión y recall [citation:5]:

$$F_\beta@k = (1+\beta^2) \cdot \frac{\text{Precision@k} \cdot \text{Recall@k}}{\beta^2 \cdot \text{Precision@k} + \text{Recall@k}}$$

Con $\beta = 1$ se obtiene F1-score, que da igual peso a ambas métricas.

Mean Average Precision (MAP)

Promedia la precisión en cada posición donde aparece un ítem relevante [citation:2]. Para un usuario, Average Precision (AP) se define como:

$$\text{AP} = \frac{\sum_{k=1}^{n} \text{Precision@k} \cdot \text{rel}(k)}{\text{número total de ítems relevantes}}$$

donde $\text{rel}(k) = 1$ si el ítem en posición k es relevante. MAP es el promedio de AP sobre todos los usuarios.

NDCG (Normalized Discounted Cumulative Gain)

Métrica que penaliza si un ítem relevante aparece en posiciones bajas del ranking [citation:2]. DCG (Discounted Cumulative Gain) se calcula como:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}$$

donde rel_i puede ser una valoración binaria o un nivel de relevancia. NDCG normaliza dividiendo por el DCG ideal (IDCG), donde los ítems relevantes están ordenados óptimamente.

Cobertura (Coverage)

Mide qué porcentaje de usuarios o ítems pueden ser recomendados [citation:2]. Un sistema que solo recomienda los mismos ítems populares tiene alta precisión pero baja cobertura.

1.6. Problemas y desafíos en sistemas de recomendación

1.6.1. Cold start

Es el problema más común y crítico [citation:3][citation:4]. Ocurre cuando no hay suficiente información para hacer recomendaciones. Tiene tres variantes:

- **Nuevo usuario (new user)**: el usuario no tiene historial de interacciones. Soluciones: pedir valoraciones iniciales en un onboarding, usar datos demográficos (edad, ubicación), o recomendar ítems populares.
- **Nuevo ítem (new item)**: el ítem no tiene valoraciones. Soluciones: usar filtrado basado en contenido (características del ítem), o combinarlo con información de metadata.
- **Nuevo sistema (new system)**: la plataforma acaba de lanzarse y no hay datos. Soluciones: transfer learning desde dominios similares, usar encuestas iniciales.

1.6.2. Escalabilidad

Con millones de usuarios e ítems, calcular similitudes entre todos los pares es inviable. Se requieren técnicas como [citation:1]:

- Indexación para búsqueda de vecinos (locality-sensitive hashing).
- Factorización matricial con SGD, que escala linealmente con el número de valoraciones.
- Modelos basados en grafos (Graph Neural Networks) para capturar relaciones complejas.

1.6.3. Matriz dispersa (sparse data)

La mayoría de los usuarios interactúan con una fracción muy pequeña de los ítems. La factorización matricial con regularización ayuda a generalizar, pero cuando la dispersión es extrema (menos de 5 valoraciones por usuario), incluso estos modelos fallan.

1.6.4. Diversidad y serendipia

Los sistemas tienden a recomendar siempre ítems muy similares, creando "filtros burbuja" [citation:4]. Es importante equilibrar precisión con diversidad y capacidad de sorprender al usuario (serendipia).

1.7. Caso integrado: recomendación de películas con MovieLens

1.7.1. Datos

Se utiliza el dataset MovieLens 100k, que contiene 100,000 valoraciones (1-5) de 943 usuarios sobre 1,682 películas [citation:7]. Cada usuario ha valorado al menos 20 películas.

1.7.2. Preprocesamiento

1. Crear matriz usuario-película con valoraciones.
2. Dividir en entrenamiento (80 %) y prueba (20 %) de forma estratificada por usuario.
3. Para cold start simulado, se pueden eliminar algunos usuarios/ítems del entrenamiento.

1.7.3. Modelos implementados

1. **Línea base**: media global + sesgos de usuario e ítem.
2. **Filtrado colaborativo basado en ítems**: similitud de coseno.
3. **Funk SVD**: con 20 factores latentes, regularización $\lambda = 0.1$, 20 épocas.

1.7.4. Métricas de evaluación

- Para predicción: RMSE y MAE en el conjunto de prueba.
- Para rankings: Precision@10, Recall@10 y NDCG@10.

1.7.5. Resultados típicos

Funk SVD supera consistentemente a los demás, confirmando su eficacia [citation:1]. La mejora en NDCG indica que además de acertar más, ordena mejor las recomendaciones.

Modelo	RMSE	MAE	Precision@10	NDCG@10
Línea base (sesgos)	1.12	0.91	0.18	0.35
Item-based CF	0.98	0.78	0.24	0.47
Funk SVD (20 factores)	0.92	0.73	0.29	0.54

Cuadro 1.1: Comparación de modelos en MovieLens 100k.

1.8. Implementación en Python

1.8.1. Filtrado colaborativo con Surprise

```
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split
from surprise import accuracy

# Cargar datos
reader = Reader(line_format='user item rating', sep='\t')
data = Dataset.load_from_file('u.data', reader=reader)
trainset, testset = train_test_split(data, test_size=0.2)

# Entrenar SVD
algo = SVD(n_factors=20, reg_all=0.1, lr_all=0.005, n_epochs=20)
algo.fit(trainset)

# Evaluar
predictions = algo.test(testset)
rmse = accuracy.rmse(predictions)
mae = accuracy.mae(predictions)
```

1.8.2. Funk SVD desde cero con NumPy

```
import numpy as np

def funk_sgd(R, k=20, lambda_reg=0.1, lr=0.005, epochs=20):
    m, n = R.shape
    P = np.random.normal(0, 0.1, (m, k))
    Q = np.random.normal(0, 0.1, (n, k))

    nonzeros = np.argwhere(~np.isnan(R))

    for epoch in range(epochs):
        np.random.shuffle(nonzeros)
        for u, i in nonzeros:
            rui = R[u, i]
            pred = np.dot(P[u], Q[i])
            eui = rui - pred
            # Actualizar
            P[u] += lr * (eui * Q[i] - lambda_reg * P[u])
            Q[i] += lr * (eui * P[u] - lambda_reg * Q[i])
    return P, Q
```

1.8.3. Cálculo de Precision@k

```
def precision_at_k(recommended_items, relevant_items, k):
    recommended_k = recommended_items[:k]
    hits = len(set(recommended_k) & set(relevant_items))
    return hits / k
```

1.9. Aplicaciones en la industria

1.9.1. Netflix

Netflix utiliza un sistema híbrido que combina filtrado colaborativo, contenido y factores contextuales [citation:4][citation:9]. Además de recomendar películas, personaliza las carátulas según el perfil del usuario: si has visto muchas películas de una actriz, aparecerá su rostro en la miniatura [citation:7].

1.9.2. Amazon

Más del 35 % de las ventas de Amazon provienen de su motor de recomendaciones [citation:9]. Utiliza principalmente filtrado colaborativo basado en ítems ("los clientes que compraron esto también compraron...") junto con reglas de negocio y popularidad.

1.9.3. Spotify

Spotify combina filtrado colaborativo con análisis de audio y playlists colaborativas [citation:4][citation:9]. Su lista "Descubrimiento Semanal" es un ejemplo clásico de sistema híbrido.

1.9.4. TikTok e Instagram

Las redes sociales utilizan modelos de deep learning que consideran interacciones, tiempo de visualización y redes de seguidos para recomendar contenido en el feed [citation:4]. La personalización es extrema y en tiempo real.

1.10. Conexión con el resto del curso

- La factorización matricial (SVD) se basa en álgebra lineal, vista en sesiones previas.
- Las métricas Precision@k y Recall@k extienden los conceptos de clasificación a rankings.
- El cold start se relaciona con el problema de datos faltantes visto en feature engineering.
- La optimización SGD en Funk SVD conecta con el entrenamiento de redes neuronales.

1.11. Resumen

- Los sistemas de recomendación se clasifican en basados en popularidad, contenido, colaborativos e híbridos.
- El filtrado colaborativo puede ser basado en usuarios o en ítems, utilizando medidas de similitud como Pearson o coseno.
- La factorización matricial (Funk SVD) con SGD y regularización es el estado del arte para datos dispersos.
- Las métricas de evaluación incluyen RMSE/MAE para ratings, y Precision@k, Recall@k, MAP, NDCG para rankings.
- El cold start es el principal desafío, abordable con estrategias híbridas y datos demográficos.
- Grandes tecnológicas como Netflix, Amazon y Spotify basan su negocio en estos sistemas, logrando incrementos significativos en ventas y retención.